

INSTITUTE OF ENGINEERING
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
KALANKI, KATHMANDU
(AFFILIATED TO TRIBHUVAN UNIVERSITY)



ADVANCED COLLEGE
OF ENGINEERING AND MANAGEMENT

LAB REPORT
SUBJECT: Distributed System
LAB NO: 2

Submitted By:

Name: Himal Joshi
Roll No: ACE079BCT028
Date: June 17, 2026

Submitted To:

**Department of Electronics and
Computer Engineering**

Implementation of Java RMI (Remote Method Invocation)

OBJECTIVES

- To understand the concept and architecture of Java Remote Method Invocation (RMI).
- To implement communication between distributed Java applications using remote objects.
- To develop a client-server application where methods are invoked remotely across a network.
- To explore the roles of RMI Registry, Remote Interface, Server Object, and Client Application.

THEORY

Java Remote Method Invocation (RMI) is a distributed computing technology that enables an object running in one Java Virtual Machine (JVM) to invoke methods on an object located in another JVM, potentially running on a different machine connected through a network. RMI extends the object-oriented paradigm beyond a single process, allowing developers to build distributed applications using familiar Java programming constructs.

Unlike traditional socket programming, RMI abstracts low-level networking details and provides a higher-level mechanism for communication between client and server applications. The client interacts with remote objects as though they were local objects, while the RMI framework transparently handles network communication, parameter transmission, and return-value delivery.

RMI is widely used in distributed systems, enterprise applications, cloud services, and network-based software where components must communicate across different machines while maintaining object-oriented design principles.

Working Principle

The Java RMI mechanism follows the following sequence of operations:

1. **Remote Interface Creation:** A remote interface is defined by extending the `java.rmi.Remote` interface. All remotely accessible methods declare `RemoteException`.
2. **Remote Object Implementation:** The server implements the remote interface

and extends `UnicastRemoteObject` to allow remote access.

3. **RMI Registry:** The server registers the remote object with the RMI Registry using a unique name.
4. **Client Lookup:** The client contacts the RMI Registry and obtains a reference to the remote object.
5. **Remote Method Invocation:** The client invokes methods on the remote object. The RMI runtime handles parameter serialization, network communication, and result transmission.
6. **Result Delivery:** The server executes the requested method and returns the result to the client.

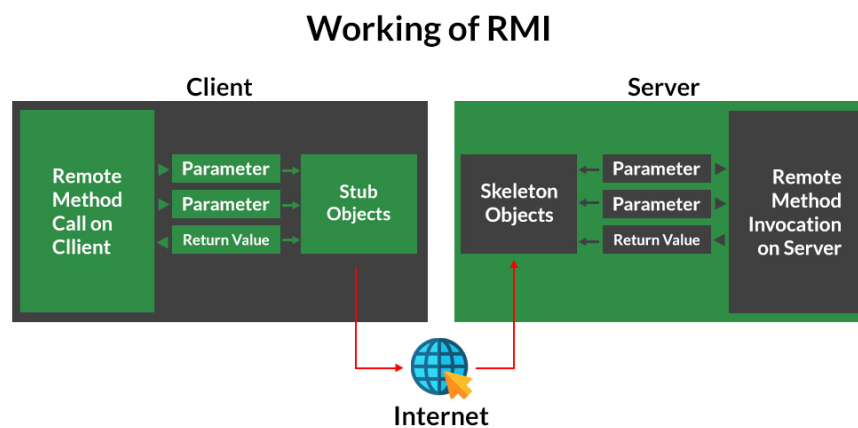


Figure 1: Implementation of Java RMI

Remote Interface

The remote interface acts as a contract between the client and server. It specifies the methods that can be called remotely. Every remote method must throw a `RemoteException` because network communication can fail due to various reasons such as connection loss or server unavailability.

The remote interface ensures that both the client and server have a common understanding of the available services.

RMI Registry

The RMI Registry is a naming service provided by Java that stores references to remote objects. Servers register remote objects with a unique name, and clients use this name to locate and access those objects.

The registry simplifies service discovery by eliminating the need for clients to know the exact network location of remote objects.

Marshalling and Unmarshalling

Marshalling is the process of converting method arguments and return values into a stream of bytes suitable for transmission across a network. Java automatically serializes objects that implement the `Serializable` interface.

Unmarshalling is the reverse process, where received byte streams are reconstructed into Java objects. These operations allow complex data structures to be exchanged between distributed applications while preserving their integrity.

Stub and Skeleton

In Java RMI, the client interacts with a local proxy known as a **stub**. The stub forwards method invocations to the remote server.

Historically, Java also used a **skeleton** on the server side to receive requests and dispatch them to the actual implementation. Modern Java versions dynamically generate these components at runtime, simplifying development and deployment.

Advantages of Java RMI

- Supports true object-oriented distributed programming.
- Simplifies network communication.
- Provides automatic serialization and deserialization.
- Supports dynamic class loading.
- Enables transparent remote method invocation.

```

1  import java.rmi.Naming;
2  import java.rmi.RemoteException;
3  import java.rmi.registry.LocateRegistry;
4  import java.rmi.server.UnicastRemoteObject;
5
6  public class RmiServer extends UnicastRemoteObject implements RmiServerIntf
   ⇨ {
7
8      public RmiServer() throws RemoteException {
9          super();
10     }
11
12     public int add(int a, int b) throws RemoteException {
13         return a + b;
14     }
15
16     public int subtract(int a, int b) throws RemoteException {
17         return a - b;
18     }
19
20     public int multiply(int a, int b) throws RemoteException {
21         return a * b;
22     }
23
24     public int divide(int a, int b) throws RemoteException {
25         if (b == 0) {
26             throw new RemoteException("Cannot divide by zero");
27         }
28         return a / b;
29     }
30
31     public static void main(String args[]) {
32         try {
33             System.out.println("RMI server started");
34
35             LocateRegistry.createRegistry(1099);
36             System.out.println("Java RMI registry created.");
37
38             RmiServer obj = new RmiServer();
39             Naming.rebind("//localhost/CalculatorService", obj);
40             System.out.println("Calculator service bound in registry");
41         } catch (Exception e) {
42             System.err.println("RMI server exception: " + e.getMessage());
43             e.printStackTrace();
44         }
45     }
46 }
47
48 OUTPUT;
49 cd "c:\Users\himal\Desktop\Lab
   ⇨ Reports\Distributed-System-Lab\LAB-2-java-calc\" ; if ($?) { javac
   ⇨ RmiServer.java } ; if ($?) { java RmiServer }
50 RMI server started
51 Java RMI registry created.
52 Calculator service bound in registry
53

```

Listing 1: *RMIserver.java*

```

1  // Source code is decompiled from a .class file using FernFlower decompiler
   ↪ (from IntelliJ IDEA).
2  import java.rmi.Naming;
3  import java.rmi.RemoteException;
4  import java.rmi.registry.LocateRegistry;
5  import java.rmi.server.UnicastRemoteObject;
6
7  public class RmiServer extends UnicastRemoteObject implements RmiServerIntf
   ↪ {
8      public RmiServer() throws RemoteException {
9      }
10
11     public int add(int var1, int var2) throws RemoteException {
12         return var1 + var2;
13     }
14
15     public int subtract(int var1, int var2) throws RemoteException {
16         return var1 - var2;
17     }
18
19     public int multiply(int var1, int var2) throws RemoteException {
20         return var1 * var2;
21     }
22
23     public int divide(int var1, int var2) throws RemoteException {
24         if (var2 == 0) {
25             throw new RemoteException("Cannot divide by zero");
26         } else {
27             return var1 / var2;
28         }
29     }
30
31     public static void main(String[] var0) {
32         try {
33             System.out.println("RMI server started");
34             LocateRegistry.createRegistry(1099);
35             System.out.println("Java RMI registry created.");
36             RmiServer var1 = new RmiServer();
37             Naming.rebind("//localhost/CalculatorService", var1);
38             System.out.println("Calculator service bound in registry");
39         } catch (Exception var2) {
40             System.err.println("RMI server exception: " + var2.getMessage());
41             var2.printStackTrace();
42         }
43     }
44 }
45
46

```

Listing 2: *RMIserver.class*

```

1  import java.rmi.Naming;
2  import java.util.Scanner;
3
4  public class RmiClient {
5      public static void main(String args[]) {
6          try {
7              RmiServerIntf obj = (RmiServerIntf)
8                  ↳ Naming.lookup("//localhost/CalculatorService");
9
10             Scanner scanner = new Scanner(System.in);
11
12             System.out.print("Enter the first number: ");
13             int a = scanner.nextInt();
14
15             System.out.print("Enter the second number: ");
16             int b = scanner.nextInt();
17
18             System.out.println("Addition result: " + obj.add(a, b));
19             System.out.println("Subtraction result: " + obj.subtract(a,
20                 ↳ b));
21             System.out.println("Multiplication result: " + obj.multiply(a,
22                 ↳ b));
23
24             try {
25                 System.out.println("Division result: " + obj.divide(a, b));
26             } catch (Exception e) {
27                 System.out.println("Division error: " + e.getMessage());
28             }
29         } catch (Exception e) {
30             System.err.println("RMI client exception: " + e.getMessage());
31             e.printStackTrace();
32         }
33     }
34 }
35
36 OUTPUT:
37 cd "c:\Users\himal\Desktop\Lab
38 ↳ Reports\Distributed-System-Lab\LAB-2-java-calc\" ; if ($?) { javac
39 ↳ RmiClient.java } ; if ($?) { java RmiClient }
40 Enter the first number: 3
41 Enter the second number: 4
42 Addition result: 7
43 Subtraction result: -1
44 Multiplication result: 12
45 Division result: 0

```

Listing 3: RMIserver.client

```

1  // Source code is decompiled from a .class file using FernFlower decompiler
   ↪ (from IntelliJ IDEA).
2  import java.io.PrintStream;
3  import java.rmi.Naming;
4  import java.util.Scanner;
5
6  public class RmiClient {
7      public RmiClient() {
8      }
9
10     public static void main(String[] var0) {
11         try {
12             RmiServerIntf var1 =
13                 ↪ (RmiServerIntf)Naming.lookup("//localhost/CalculatorService");
14             Scanner var2 = new Scanner(System.in);
15             System.out.print("Enter the first number: ");
16             int var3 = var2.nextInt();
17             System.out.print("Enter the second number: ");
18             int var4 = var2.nextInt();
19             PrintStream var10000 = System.out;
20             int var10001 = var1.add(var3, var4);
21             var10000.println("Addition result: " + var10001);
22             var10000 = System.out;
23             var10001 = var1.subtract(var3, var4);
24             var10000.println("Subtraction result: " + var10001);
25             var10000 = System.out;
26             var10001 = var1.multiply(var3, var4);
27             var10000.println("Multiplication result: " + var10001);
28
29             try {
30                 var10000 = System.out;
31                 var10001 = var1.divide(var3, var4);
32                 var10000.println("Division result: " + var10001);
33             } catch (Exception var6) {
34                 System.out.println("Division error: " + var6.getMessage());
35             }
36         } catch (Exception var7) {
37             System.err.println("RMI client exception: " + var7.getMessage());
38             var7.printStackTrace();
39         }
40     }
41 }
42

```

Listing 4: *RmiClient.class*

DISCUSSION

This laboratory exercise demonstrated the implementation of Java Remote Method Invocation (RMI) for communication between distributed applications. A client-server architecture was developed in which arithmetic operations were executed on a remote server while being invoked transparently from the client application.

The experiment highlighted several important distributed-system concepts including remote object references, service registration, object serialization, and network transparency. Through the use of RMI Registry, clients were able to dynamically discover remote services without requiring direct knowledge of server implementation details.

Java RMI significantly simplified distributed application development by automatically managing communication, parameter passing, and response handling. Compared to socket programming, RMI reduced programming complexity while preserving an object-oriented design.

However, certain limitations exist. Network failures can interrupt communication, requiring robust exception handling. Performance overhead may occur due to serialization and network latency. Additionally, RMI primarily supports Java-to-Java communication, making interoperability with other programming languages more challenging than technologies such as REST APIs or gRPC.

CONCLUSION

The laboratory successfully demonstrated the implementation of Java RMI and validated the concept of remote method invocation in distributed systems. The client application successfully invoked arithmetic methods hosted on a remote server through the RMI framework.

The experiment provided practical experience with remote interfaces, RMI Registry, object serialization, and distributed object communication. Java RMI proved to be an effective mechanism for building distributed Java applications while abstracting the complexities of underlying network communication.

Future enhancements could include implementing authentication, secure communication using SSL, support for complex object transfer, and deployment across multiple physical machines to create larger distributed systems.